

Aufgabe 2: Dreieckspuzzle

Team-ID: 00048

Team: while(!success){try();}

Bearbeiter/-innen dieser Aufgabe:
Rui Zhang

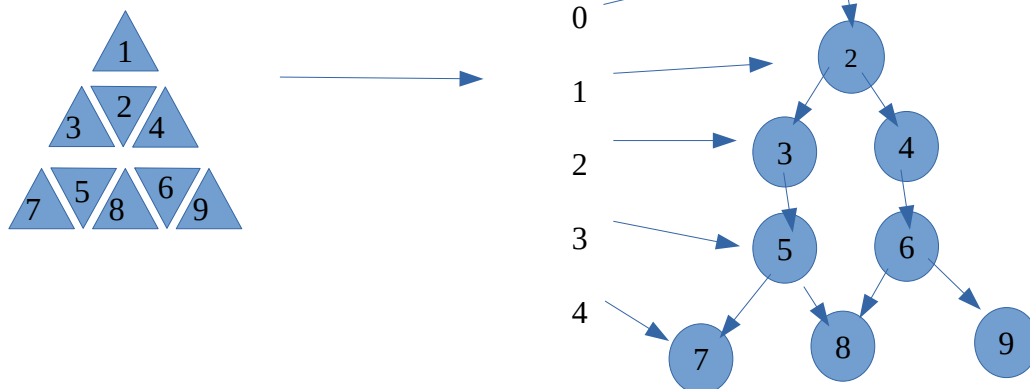
21. November 2020

Inhaltsverzeichnis

Lösungsidee.....	1
Umsetzung.....	2
Beispiele.....	4
Quellcode.....	4

Lösungsidee

Die grundlegende Idee der Lösung ist, dass ein Dreieckspuzzle aus „Schichten“ besteht und die Dreiecke, die in einer Schicht eingesetzt werden können, von den Dreiecken und deren Ausrichtungen in der vorherigen Schicht abhängig ist.



Die einzelnen Schichten werden in separate Lists gespeichert, wo der Index dann genutzt werden kann, um die „Eltern“ eines Knotens herauszufinden.

Ein rekursiver Algorithmus soll also das Dreieckspuzzle lösen, indem er jede Schicht durchgeht und alle möglichen Dreiecke, die man in dieser Schicht, in jede Position einsetzen kann, ermittelt. Das wird anhand der sogenannten „Eltern“ also die Dreiecke von der vorherigen Schicht gemacht.

Danach werden alle Kombinationen der Dreiecke, die man in dieser Schicht einsetzen kann, nacheinander ausprobiert und abgefragt. Dabei wird überprüft, ob die nächste Schicht überhaupt eine Lösung besitzen kann (ob die übrigen Dreiecke überhaupt passen). Falls nicht, dann wird die nächste Kombination ausprobiert. Falls ja, dann wird die jetzige Kombination in die Lösung eingesetzt. Dann wird der Methode (dieselbe Methode, weil der Algorithmus rekursiv ist), welche die letzte Schicht behandelt hat, mitgeteilt, dass diese Schicht eine Lösung gefunden hat, die passt. Der letzte Methodenaufruf macht das Gleiche und so weiter.

Umsetzung

In dem eigentlichen Programm gibt es drei Klassen, welche für das Lösen des Programms relevant sind:

Die Klasse „Dreiecke“ speichert die Figurenhälften ab, die das Dreieck besitzt und eine ID zur Identifizierung des Dreiecks ab.

„Dreiecksnodes“ (Dreiecksknoten) werden bei jeder Rekursion erstellt und speichern ihre Position in der Schicht ab. Dasselbe geschieht mit allen möglichen Dreiecken, die man in dieser Position einsetzen kann. Dreiecke können in Dreiecksknoten eingesetzt werden und ebenfalls die Seiten des Dreiecks, die zur nächsten Schicht zeigen, auch. Das Einsetzen eines Dreiecks und das Bestimmen der Seiten, die angezeigt werden, passiert aber erst in der Phase, in welcher die Kombinationen der Dreiecke ausprobiert wird.

Die Klasse „Dreieckspuzzle“ besitzt die Methoden zur Bestimmung der „Eltern“ und den rekursiven Algorithmus zur Bestimmung der Lösung sowie weitere Hilfsmethoden.

„Pair“ ist eine Klasse zur Erstellung von Schlüssel-Werte-Paaren.

Am Anfang des Programms wird ein Objekt vom Typ DreiecksPuzzle erstellt. Davor wird der Nutzer abgefragt, welche Beispieldatei gelöst werden soll. Das Dreieckspuzzle liest die gegebene Datei ein, wobei eine Liste von Dreiecken und eine HashMap erstellt werden. Letztere enthält nach dem Lösen des Puzzles die Lösung.

Das Objekt ruft dann die Methode „loesePuzzle()“ auf. Diese geht mithilfe von zwei verschachtelten for-loops alle Dreiecke und alle Ausrichtungen des Dreiecks durch. Am Ende jedes Schleifenablaufs wird die Methode „naechsteSchicht()“ aufgerufen, welche rekursiv die Lösungen der folgenden Schichten (falls es überhaupt eine Lösung gibt) bestimmt. Falls es keine Lösung gibt, dann gibt die Methode „false“ zurück, falls sie aber eine Lösung gefunden wird, dann gibt sie „true“ zurück. Somit weiß das Programm, ob es überhaupt eine Lösung für das Puzzle gibt oder nicht.

Die „naechsteSchicht()“ Methode braucht folgenden Informationen, die mittels Parameter überreicht werden:

- die Schichtzahl (Der Index dieser Schicht)
- alle übrigen Dreiecke, die noch nicht genutzt wurden
- die vorherige Schicht

Bevor überhaupt etwas in der Methode geschieht, wird abgefragt, ob noch Dreiecke übrig sind. Schließlich ist das Programm an der letzten Schicht angekommen, falls dies der Fall ist und der Algorithmus muss dann terminieren. Dies ist also die Abbruchsbedingung. Die Methode gibt „true“ zurück, um zu signalisieren, dass eine Lösung gefunden wurde.

„naechsteSchicht()“ berechnet mit der Schichtzahl die Anzahl der Dreiecksknoten, die in diese Schicht eingesetzt werden müssen und setzt auch entsprechend viele ein. Dabei werden die Dreiecksknoten in einer for-Schleife erstellt, wobei nach jeder Iteration alle Eltern dieses Knotens mithilfe von „getEltern()“ herausgefunden werden. Danach werden alle Dreiecke, die in dem jetzigen Knoten eingesetzt werden können, durch „getAlleMoeglichenWerteFuerKnoten()“ ermittelt. Diese Methode bekommt die Elternknoten, die die „getEltern()“ Methode gefunden hat. Schließlich sind die Werte der Knoten abhängig von den Dreiecken der Elternknoten aufgrund der Figurenhälften.

Am Ende wird dann eine weitere Methode „probiereAlleKombinationen()“ angewendet (die auch rekursiv ist), um alle Kombinationen aller einsetzbaren Dreiecke in den jeweiligen Knoten dieser Schicht auszuprobieren. Nach dem erstellen jeder Kombination wird die „naechsteSchicht()“ Methode aufgerufen, um abzufragen, ob die jetzige Kombination eine Lösung für die folgenden Schichten anbietet (einfacher formuliert: Ob die übrig gelassenen Dreiecke für eine Lösung der restlichen Schichten geeignet sind.). Falls nicht, so wird die nächste Kombination ausprobiert. Falls keine einzige Kombination eine geeignete Lösung darstellt, dann wird „false“ zurückgegeben und der jetzige Aufruf von „naechsteSchicht()“ gibt somit auch „false“ zurück, wodurch der vorherige Aufruf von „naechsteSchicht()“ dann edie nächste Kombination für die vorherige Schicht ausprobiert. Wenn jedoch die jetzige Kombination eine Lösung für die folgenden Schichten anbietet, dann ist eine Lösung für das Dreieckspuzzle gefunden worden und die jetzige Kombination wird in die HashMap „alleSchichten“ eingesetzt. Diese soll schließlich die entgültige Lösung des Puzzles speichern. Danach wird „true“ zurückgegeben, sodass die vorherige Schichten in die Lösungs-HashMap eingesetzt werden können.

Die „getEltern()“ Methode und die „getAlleMoeglichenWerteFuerKnoten()“ werden im Folgenden erläutert.

Die „getEltern()“ Methode nutzt die Position des Knotens in seiner Schicht (der Index in der Liste (siehe Lösungs idee)) und die Liste der Knoten in der vorherigen Schicht und erstellt somit eine Liste von Elternknoten. Dies geschieht, indem zwischen einer „geraden Schicht“ (Schicht mit gerader Zahl) und einer „ungeraden Schicht“ (Schicht mit ungerader Zahl) unterschieden wird. Laut der Abbildung besitzen alle Knoten in einer ungeraden Schicht nur einen Elternknoten, die Knoten in einer geraden Schicht jedoch entweder zwei oder auch nur einen Elternknoten. Durch if-

statements wird zwischen diesen Fällen unterschieden und die Elternknoten werden zu der Liste hinzugefügt.

Die „getAlleMoeglichenWerteFuerKnoten()“ Methode probiert schließlich alle übrigen Dreiecke und alle Ausrichtungen dessen aus. Falls die Figurenhälften des jetzigen Dreiecks zu den Figurenhälften der Dreiecke der Elternknoten passen, dann wird die Seite des jetzigen Dreiecks, welche in Richtung der nächsten Schicht zeigt, berechnet. Diese Seite und das Dreieck wird als Paar zu einer Liste hinzugefügt und das nächste Dreieck bzw. die nächste Ausrichtung wird ausprobiert. Am Ende wird die hier genannte Liste, die jetzt alle möglichen Dreiecken und Ausrichtungen als Paare enthält, zurückgegeben.

Beispiele

Die Beispiele aus der BWINF Webseite wurden benutzt, um das Programm auszutesten.

Die Beispiele 0 und 1 besaßen beide nur 3 verschiedene Figuren. Beispiele 2 und 3 jedoch mehr.

Beispiel 3 ist jedoch besonders, denn sie ist besonders leicht für den Mensch zu lösen. Nach näherer Betrachtung wird es deutlich, dass es zwar 10 Figuren gibt, jedoch keine untere Hälften für diese Figur, weil in der Datei die Zahl -10 nie auftaucht. Also müssen alle Dreiecke, die die Figurenhälfte 10 besitzen, nach außen positioniert werden. Die restlichen Dreiecke kann man entsprechend der Figurenhälften der äußeren Dreiecke einsetzen.

Dieses Beispiel eignet sich somit gut als Testbeispiel für das Programm, denn wenn das Programm bei diesem Beispiel keine Lösung ausgibt, dann liegt bestimmt ein Logikfehler vor. Andere Beispiele sind nicht besonders einfach zu lösen, sind von daher weniger geeignet. Schließlich kann man die Lösung des Programms nicht überprüfen.

Für diese Lösungsmethode ist die Anzahl der Figurenhälften, die vorhanden sind, irrelevant. Also stellen Beispiele 2 und 3 keine Sonderfälle dar.

Quellcode

```
public class DreiecksPuzzle {  
    private final ArrayList<Dreieck> teile;  
    private final HashMap<Integer, ArrayList<DreiecksNode>> alleSchichten;  
  
    public DreiecksPuzzle(String path) throws IOException, NoSuchElementException {  
        BufferedReader reader = new BufferedReader(new  
        InputStreamReader(getClass().getResourceAsStream(path), StandardCharsets.UTF_8));  
  
        reader.readLine();
```

```
reader.readLine();
```

```
this.teile = new ArrayList<>();
```

```
this.alleSchichten = new HashMap<>();
```

```
String zeile;
```

```
while ((zeile = reader.readLine()) != null) {
```

```
    this.teile.add(new Dreieck(zeile));
```

```
}
```

```
}
```

```
public HashMap<Integer, ArrayList<DreiecksNode>> getAlleSchichten() { return  
alleSchichten; }
```

```
public ArrayList<Dreieck> getTeile() { return teile; }
```

```
public boolean loesePuzzle() {
```

```
    ArrayList<Dreieck> uebrigeTeile = new ArrayList<>(this.teile);
```

```
    for (Dreieck teil : uebrigeTeile) {
```

```
        DreiecksNode jetzigerDreiecksknoten = new DreiecksNode(teil);
```

```
        jetzigerDreiecksknoten.setPositionInSchicht(0);
```

```
        for (int seite=0; seite<3; seite++) {
```

```
            ArrayList<Integer> offeneSeiten = new ArrayList<>();
```

```
            offeneSeiten.add(seite);
```

```
            jetzigerDreiecksknoten.setOffeneSeiten(offeneSeiten);
```

```
        int jetzigeSchichtzahl = 0;
```

```
        ArrayList<DreiecksNode> schicht0 = new ArrayList<DreiecksNode>() {{
```

```
            add(jetzigerDreiecksknoten);
```

```
}};
```

```
ArrayList<Dreieck> neueUebrigTeile = new ArrayList<>(uebrigTeile);
```

```
neueUebrigTeile.remove(teil);
```

```
if (this.naechsteSchicht(jetzigSchichtzahl + 1, neueUebrigTeile, schicht0)) {
```

```
    this.alleSchichten.put(jetzigSchichtzahl, schicht0);
```

```
    return true;
```

```
}
```

```
}
```

```
}
```

```
return false;
```

```
}
```

```
private ArrayList<DreiecksNode> getEltern(int schichtZahl, int positionInSchicht,  
ArrayList<DreiecksNode> vorherigeSchicht) {
```

```
    ArrayList<DreiecksNode> eltern = new ArrayList<>();
```

```
    if (schichtZahl % 2 == 0) {
```

```
        if (positionInSchicht - 1 >= 0) {
```

```
            DreiecksNode eltern0 = vorherigeSchicht.get(positionInSchicht - 1);
```

```
            eltern.add(eltern0);
```

```
        }
```

```
        if (positionInSchicht < vorherigeSchicht.size()) {
```

```
            DreiecksNode eltern1 = vorherigeSchicht.get(positionInSchicht);
```

```
            eltern.add(eltern1);
```

```
        }
```

```
    } else {
```

```
        eltern.add(vorherigeSchicht.get(positionInSchicht));
```

```
    }
```

```
    return eltern;
```

```
}
```

```
private ArrayList<Pair<Dreieck, ArrayList<Integer>>>>
getAlleMoeglichenWerteFuerKnoten(ArrayList<DreiecksNode> elternteile, ArrayList<Dreieck>
uebrigeTeile, int positionInSchicht) {

    ArrayList<Pair<Dreieck, ArrayList<Integer>>>> moeglicheWerte = new ArrayList<>();

    for (Dreieck teil : uebrigeTeile) {
        if (elternteile.size() == 1) {
            DreiecksNode dreiecksknotenVonElternteil = elternteile.get(0);

            ArrayList<Integer> offeneSeitenVonEltern =
dreiecksknotenVonElternteil.getOffeneSeiten();

            int offeneSeite = offeneSeitenVonEltern.get(positionInSchicht -
dreiecksknotenVonElternteil.getPositionInSchicht());

            int offeneFigurenHaelften =
dreiecksknotenVonElternteil.getWert().getFigurenHaelften().get(offeneSeite);
            ArrayList<Integer> figurenHaelftenVonDreieck = teil.getFigurenHaelften();

            for (int seite = 0; seite < 3; seite++) {
                if (figurenHaelftenVonDreieck.get(seite) == offeneFigurenHaelften * -1) {

                    ArrayList<Integer> alleOffeneSeiten = new ArrayList<>();

                    if (dreiecksknotenVonElternteil.getPositionInSchicht() == positionInSchicht) {
                        if (offeneSeitenVonEltern.size() == 2) {
                            alleOffeneSeiten.add(seite + 1 - (Math.floorDiv(seite, 2) * 3));

                        } else {
                            int rechteOffeneSeite = seite + 1 - (Math.floorDiv(seite, 2) * 3);
                            int linkeOffeneSeite = seite + 2 - (Math.floorDiv(seite + 1, 2) * 3);
```

```
        alleOffeneSeiten.add(linkOffeneSeite);
        alleOffeneSeiten.add(rechteOffeneSeite);

    }
} else if (dreiecksknotenVonElternteil.getPositionInSchicht() == positionInSchicht -
1) {

    int neueOffeneSeite = seite - 1;
    if (neueOffeneSeite < 0) {
        neueOffeneSeite += 3;
    }
    alleOffeneSeiten.add(neueOffeneSeite);
}

Pair<Dreieck, ArrayList<Integer>> wertePaare = new Pair<>(teil, alleOffeneSeiten);
moeglicheWerte.add(wertePaare);
}
}

} else if (elternteile.size() == 2) {

    DreiecksNode elternteil0 = elternteile.get(0);
    DreiecksNode elternteil1 = elternteile.get(1);

    int linkesElternteilOffeneSeite = elternteil0.getOffeneSeiten().get(1);
    int rechtesElternteilOffeneSeite = elternteil1.getOffeneSeiten().get(0);

    int linkesElternteilOffeneFigurenHaelfte =
    elternteil0.getWert().getFigurenHaelften().get(linkesElternteilOffeneSeite);

    int rechtesElternteilOffeneFigurenHaelfte =
    elternteil1.getWert().getFigurenHaelften().get(rechtesElternteilOffeneSeite);

    ArrayList<Integer> figurenHaelftenVonDreieck = teil.getFigurenHaelften();

    for (int seiteLinks = 0, seiteRechts = 1; seiteLinks < 3; seiteLinks++, seiteRechts++) {
```



```
        if (seiteRechts == 3) {
            seiteRechts -= 3;
        }

        if ((figurenHaelftenVonDreieck.get(seiteLinks) ==
linkesElternteilOffeneFigurenHaelfte * -1) &&
            (figurenHaelftenVonDreieck.get(seiteRechts) ==
rechtesElternteilOffeneFigurenHaelfte * -1)) {

            ArrayList<Integer> alleOffeneSeiten = new ArrayList<>();
            alleOffeneSeiten.add(2 - (seiteLinks + seiteRechts - 1));

            Pair<Dreieck, ArrayList<Integer>> currentValuePair = new Pair<>(teil,
alleOffeneSeiten);
            moeglicheWerte.add(currentValuePair);
        }
    }
}

return moeglicheWerte;
}

private boolean probiereAlleKombinationen(ArrayList<DreiecksNode> jetzigeKombination,
ArrayList<DreiecksNode> jetzigeSchicht, int schichtZahl, ArrayList<Dreieck> uebrigeTeile) {
    DreiecksNode jetzigerKnoten = jetzigeSchicht.get(0);
    ArrayList<DreiecksNode> neueSchicht = new ArrayList<>(jetzigeSchicht);
    neueSchicht.remove(0);

    for (Pair<Dreieck, ArrayList<Integer>> wertepaar:
jetzigerKnoten.getAlleMoeglichenDreiecke()) {

        Dreieck dreieckVonWertepaar = wertepaar.getKey();
```

```
    if (uebrigeTeile.contains(dreieckVonWertePaar)) {
        ArrayList<DreiecksNode> neueKombination = new ArrayList<>(jetzigeKombination);
        neueKombination.add(new DreiecksNode(jetzigerKnoten.getPositionInSchicht(),
dreieckVonWertePaar, wertepaar.getValue()));
        ArrayList<Dreieck> neueUebrigeTeile = new ArrayList<>(uebrigeTeile);
        neueUebrigeTeile.remove(dreieckVonWertePaar);

        if (neueSchicht.size() == 0) {
            if (this.naechsteSchicht(schichtZahl + 1, neueUebrigeTeile, neueKombination)) {
                this.alleSchichten.put(schichtZahl, neueKombination);
                return true;
            }
        } else {
            return probiereAlleKombinationen(neueKombination, neueSchicht, schichtZahl,
neueUebrigeTeile);
        }
    }
    return false;
}

private boolean naechsteSchicht(int schichtZahl, ArrayList<Dreieck> uebrigeTeile,
ArrayList<DreiecksNode> vorherigeSchicht) {
    if (uebrigeTeile.size() == 0) {
        return true;
    }

    ArrayList<DreiecksNode> jetzigeSchicht = new ArrayList<>();
    int anzahlDerKnoten = (int) Math.ceil((double) (schichtZahl+1) / 2);
    for (int i=0; i<anzahlDerKnoten; i++) {
        DreiecksNode jetzigerDreiecksKnoten = new DreiecksNode();
```

```
    jetzigerDreiecksKnoten.setPositionInSchicht(i);

    ArrayList<DreiecksNode> elternVonDreiecksKnoten = this.getEltern(schichtZahl, i,
    vorherigeSchicht);

    ArrayList<Pair<Dreieck, ArrayList<Integer>>> alleMoeglichenWerteFuerKnoten =
    this.getAlleMoeglichenWerteFuerKnoten(elternVonDreiecksKnoten, uebrigeTeile, i);

    jetzigerDreiecksKnoten.setAlleMoeglichenDreiecke(alleMoeglichenWerteFuerKnoten);
    jetzigeSchicht.add(jetzigerDreiecksKnoten);

}

return probiereAlleKombinationen(new ArrayList<>(), jetzigeSchicht, schichtZahl,
uebrigeTeile);
}
}

public class DreiecksNode {
    private Dreieck wert;
    private ArrayList<Pair<Dreieck, ArrayList<Integer>>> alleMoeglichenDreiecke;
    private int positionInSchicht;
    private ArrayList<Integer> offeneSeiten;

    public DreiecksNode(Dreieck wert) {
        this();
        this.wert = wert;
    }

    public DreiecksNode() {
        this.positionInSchicht = -1;
        this.offeneSeiten = null;
        this.wert = null;
        this.alleMoeglichenDreiecke = null;
    }
}
```

```
}  
  
public DreiecksNode(int positionInSchicht, Dreieck wert, ArrayList<Integer> offeneSeiten) {  
    this();  
    this.positionInSchicht = positionInSchicht;  
    this.wert = wert;  
    this.offeneSeiten = offeneSeiten;  
}  
  
public Dreieck getWert() { return wert; }  
public ArrayList<Integer> getOffeneSeiten() { return this.offeneSeiten; }  
public void setOffeneSeiten(ArrayList<Integer> seiten) { this.offeneSeiten = seiten; }  
public int getPositionInSchicht() { return this.positionInSchicht; }  
public void setPositionInSchicht(int index) { this.positionInSchicht = index; }  
public ArrayList<Pair<Dreieck, ArrayList<Integer>>> getAlleMoeglichenDreiecke() { return  
this.alleMoeglichenDreiecke; }  
  
public void setAlleMoeglichenDreiecke(ArrayList<Pair<Dreieck, ArrayList<Integer>>>  
alleMoeglichenDreiecke) { this.alleMoeglichenDreiecke = alleMoeglichenDreiecke; }  
}  
  
public class Dreieck {  
    private static int IDZaehler;  
    static { IDZaehler = 1; }  
  
    private final ArrayList<Integer> figurenHaelften;  
    private final int id;  
  
    public Dreieck(String zeile) {  
        this.figurenHaelften = new ArrayList<>();  
        for (String part : zeile.trim().split(" ")) {  
            this.figurenHaelften.add(Integer.parseInt(part));  
        }  
    }  
}
```

```
    this.id = IDZaehler;  
    IDZaehler++;  
}
```

```
@Override
```

```
public boolean equals(Object o) {  
    if (this == o) return true;  
    if (o == null || getClass() != o.getClass()) return false;  
    Dreieck dreieck = (Dreieck) o;  
    return id == dreieck.id;  
}
```

```
@Override
```

```
public int hashCode() {  
    return Objects.hash(id);  
}
```

```
public int getId() { return id; }
```

```
public ArrayList<Integer> getFigurenHaelften() { return this.figurenHaelften; }
```

```
}
```

```
public class Pair<K, V> {
```

```
    private final K key;
```

```
    private final V value;
```

```
    Pair(K key, V value) {
```

```
        this.key = key;
```

```
        this.value = value;
```

```
    }
```

```
    public K getKey() {
```

```
    return key;  
}
```

```
public V getValue() {  
    return value;  
}  
}
```